🔌 Socket.IO & WebSocket Revision

This section is intentionally detailed so that this README can also be used as a revision note for Socket.IO and real-time communication.

🌐 What is a Socket?

A socket is a communication endpoint that allows two systems to communicate with each other.

In a normal web application, communication usually looks like:

```
Client
   |
   | HTTP Request
   ↓
Server
   |
   | HTTP Response
   ↓
Client
```

The client sends a request and the server responds.

For many applications, this is enough.

But consider a multiplayer chess game.

If Player 1 moves a piece:

```
Player 1 → e2 to e4
```

Player 2 should immediately see:

```
e2 → e4
```

without refreshing the page.

For this kind of application, we need real-time communication.

That is where WebSockets and Socket.IO become useful.

🔄 What is WebSocket?

WebSocket is a communication protocol that allows a persistent, two-way connection between a client and a server.

Instead of repeatedly creating new HTTP requests, the client and server maintain a connection.

```
        Persistent Connection
Client  <====================>  Server
```

Both sides can send data whenever required.

This is called full-duplex communication.

Full-duplex means:

The client can send data to the server:

```
Client → Server
```

and the server can send data to the client:

```
Client ← Server
```

at any time.

HTTP vs WebSocket

Traditional HTTP

```
Client → Request → Server
Client ← Response ← Server
```

The client generally has to initiate communication.

WebSocket

```
Client ←──────────────→ Server
       Persistent
       connection
```

After the connection is established, either side can communicate.

This makes WebSockets useful for:

- Multiplayer games
- Chat applications
- Live notifications
- Stock/live price updates
- Collaborative editors
- Live dashboards
- Real-time tracking
- Online gaming

What is Socket.IO?

Socket.IO is a JavaScript library that makes real-time, event-based communication easier.

It provides an abstraction over real-time transports, including WebSocket when available, along with additional features such as automatic reconnection and event-based APIs.

Important distinction:

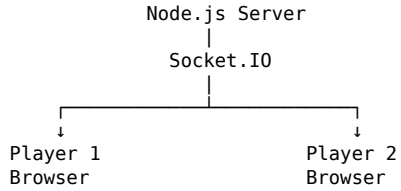
```
WebSocket = communication protocol

Socket.IO = library/framework built for real-time communication
```

Socket.IO is not the same thing as WebSocket.

It can use WebSocket as a transport, but it also provides additional functionality and its own protocol.

Socket.IO Architecture

In this project, there are three main components:



The server acts as the central point through which game events are processed.

Establishing a Socket Connection

On the frontend:

```
const socket = io();
```

This connects the browser to the Socket.IO server.

For example, in this project:

```
const socket = io();
```

means:

Connect this browser to the Socket.IO server.

Receiving a Connection on the Server

On the backend:

```
io.on("connection", function(socket) {
    console.log("New user connected:", socket.id);
});
```

Whenever a new client connects, the `connection` event is triggered.

```
Browser
   |
   | socket connection
   ↓
Server
   |
   ↓
"connection" event
```

Socket ID

Every connected client gets a unique socket ID.

Example:

```
Player 1 → ABC123
Player 2 → XYZ456
Player 3 → PQR789
```

The server can use this ID to identify which client sent a particular event.

In the chess project:

```
socket.id
```

is used to identify the connected player.

Events in Socket.IO

Socket.IO communication is primarily event-based.

An event has:

```
Event Name + Data
```

For example:

```
socket.emit("move", move);
```

Here:

```
"move" = event name
move   = data
```

emit()

`emit()` is used to send an event.

Example:

```
socket.emit("move", move);
```

This means:

Send the `"move"` event along with the `move` data.

Think:

```
emit() = SEND
```

on()

`on()` is used to listen for an event.

Example:

```
socket.on("move", function(move) {
    console.log(move);
});
```

This means:

Whenever a "move" event arrives, execute this function.

Think:

```
on() = LISTEN / RECEIVE
```

Easy Way to Remember

```
emit() → I am sending something

on()   → I am listening for something
```

Example:

```
// Send
socket.emit("message", "Hello");

// Receive
socket.on("message", function(data) {
    console.log(data);
});
```

socket.emit() vs io.emit()

This is VERY important.

socket.emit()

Sends an event to a specific socket/client.

```
socket.emit("playerRole", "w");
```

Meaning:

```
Server
  |
  | only to this socket
  ↓
One specific client
```

In this chess project, the server uses it to assign a player role:

```
socket.emit("playerRole", "w");
```

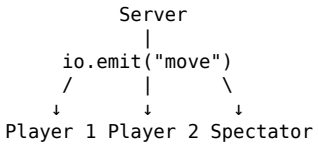
Only that particular client receives the event.

io.emit()

Broadcasts an event to all connected clients.

```
io.emit("move", move);
```

Meaning:



This is useful for the chess game because when one player makes a move, everyone should see it.

⚡ Important Socket.IO Methods

```
socket.emit()
    ↓
Send to this socket

socket.on()
    ↓
Listen for an event

io.emit()
    ↓
Send to all connected sockets

socket.broadcast.emit()
    ↓
Send to everyone except the sender
```

□ socket.broadcast.emit()

Suppose Player 1 sends a message.

```
socket.broadcast.emit("message", data);
```

The message goes to:

```
Player 2
Player 3
Player 4
...
```

but NOT Player 1.

Therefore:

```
io.emit()
       ↓
Everyone including sender

socket.broadcast.emit()
       ↓
Everyone except sender
```

♟ How Socket.IO Works in This Chess Project

Suppose White moves:

```
e2 → e4
```

The complete flow is:

```
            PLAYER 1
               |
               | Drag piece
               ↓
          handleMove()
               |
               | socket.emit("move", move)
               ↓
            SERVER
               |
               | Validate move
               ↓
           Chess.js
               |
               | Valid
               ↓
        io.emit("move")
           /    |    \
          ↓     ↓     ↓
       White  Black  Spectator
          |     |       |
          ↓     ↓       ↓
     renderBoard()
```

1 Client Creates the Move

The frontend creates:

```
const move = {
    from: "e2",
    to: "e4"
};
```

This represents:

```
From → e2
To   → e4
```

2 Client Sends the Move

The client sends the event:

```
socket.emit("move", move);
```

So:

```
Browser
   |
   | "move" event
   ↓
Server
```

3 Server Listens for the Move

The server has:

```
socket.on("move", function(move) {
    ...
});
```

Now the server receives:

```
{
    from: "e2",
    to: "e4"
}
```

4 Server Validates the Move

The server uses Chess.js:

```
const result = chess.move(move);
```

Chess.js checks whether the move follows chess rules.

For example:

```
e2 → e4
```

is valid.

But something like:

```
e2 → e8
```

would not be a normal legal pawn move.

Why Validate on the Server?

This is an important backend concept.

Never completely trust the client.

A malicious client could send:

```
{
    from: "e2",
    to: "e8"
}
```

or attempt to move when it is not their turn.

Therefore:

```
Client
   |
   | Request
   ↓
Server
   |
   | Validate
   ↓
Game Logic
```

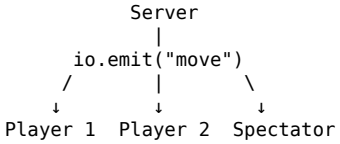
The server should be the source of truth.

5 Server Broadcasts the Valid Move

After validating:

```
io.emit("move", move);
```

The server broadcasts the move to every connected client.



6 Clients Receive the Move

Each client listens:

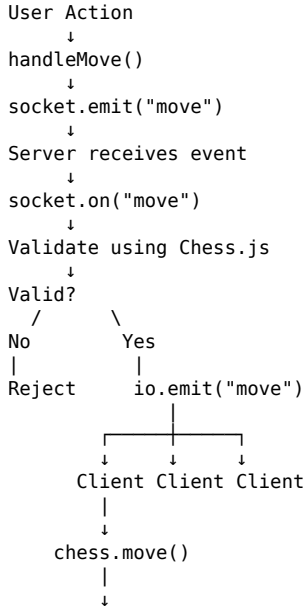
```
socket.on("move", function(move) {
    chess.move(move);
    renderBoard();
});
```

The client:

1. Receives the move
2. Updates its local Chess.js state
3. Renders the new board

Complete Move Lifecycle

Remember this:



```
    renderBoard()
```

♟ Player Assignment Using Sockets

The server maintains players:

```
let players = {};
```

When a new client connects:

```
1st client → White
2nd client → Black
3rd client → Spectator
4th client → Spectator
...
```

The server can do:

```
if (!players.white) {
    players.white = socket.id;
    socket.emit("playerRole", "w");
}
else if (!players.black) {
    players.black = socket.id;
    socket.emit("playerRole", "b");
}
else {
    socket.emit("playerRole", "spectator");
}
```

The important concept here is:

```
socket.id
    ↓
Identify client
    ↓
Assign role
```

♟ Receiving the Player Role

Frontend:

```
socket.on("playerRole", function(role) {
    playerRole = role;
    renderBoard();
});
```

The server might send:

```
"w"
```

or:

```
"b"
```

or:

```
"spectator"
```

The frontend then knows what type of user is connected.

Controlling Turns

The server should also verify whose turn it is.

Conceptually:

```
Is it White's turn?
       |
       ↓
Is this the White player's socket?
       |
      Yes
       ↓
Allow move
```

Otherwise:

```
Reject move
```

This prevents the client from simply changing the frontend and moving the opponent's pieces.

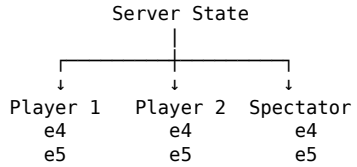
State Synchronization

One of the biggest concepts in multiplayer applications is state synchronization.

Suppose the server's state is:

```
White pawn → e4
Black pawn → e5
```

All clients should eventually have the same state.



The server acts as the central authority.

Source of Truth

In a multiplayer application:

```
Client = UI / user interaction

Server = trusted game state
```

The client can request:

```
"Move e2 → e4"
```

But the server decides:

```
"Is this move allowed?"
```

This is a very important backend principle.

🔌 Socket Connection Lifecycle

A socket connection generally follows:

```
Client starts
     ↓
Connection established
     ↓
Server assigns socket.id
     ↓
Client and server exchange events
     ↓
Real-time communication
     ↓
Client disconnects
```

Socket.IO also provides connection lifecycle events.

For example:

```
socket.on("disconnect", function() {
    console.log("User disconnected");
});
```

This can be used to clean up resources or update player state.

🚪 What Happens When a Player Disconnects?

Suppose:

```
Player 1 = White
Player 2 = Black
```

If White disconnects:

```
White socket disconnects
        ↓
Server detects disconnect
        ↓
Remove White player
        ↓
Another player can potentially take the role
```

This is why handling:

```
socket.on("disconnect", ...)
```

is important in multiplayer applications.

🧠 Event-Driven Architecture

Socket.IO applications are usually event-driven.

Instead of continuously asking:

```
"Did something happen?"
"Did something happen?"
"Did something happen?"
```

the application waits for an event.

For example:

```
socket.on("move", ...)
```

means:

When a move happens, execute this function.

This is similar to:

```
Event
  ↓
Listener
  ↓
Handler
```

□ Socket.IO Cheat Sheet

Connect client

```
const socket = io();
```

Server connection

```
io.on("connection", (socket) => {
});
```

Send event from client

```
socket.emit("eventName", data);
```

Receive event

```
socket.on("eventName", (data) => {
});
```

Send to one specific client

```
socket.emit("eventName", data);
```

Send to everyone

```
io.emit("eventName", data);
```

Send to everyone except sender

```
socket.broadcast.emit("eventName", data);
```

Detect disconnect

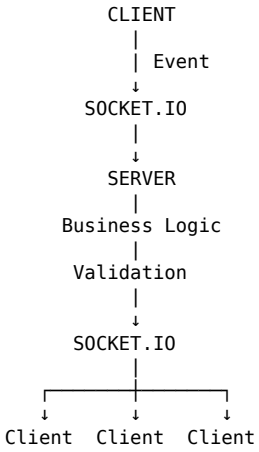
```
socket.on("disconnect", () => {
});
```

⚡ Quick Difference Table

MethodMeaning	
`socket.emit()`	Send to a particular socket
`socket.on()`	Listen for an event
`io.emit()`	Send to all connected clients
`socket.broadcast.emit()`	Send to everyone except sender
`io.on("connection")`	Detect a new client connection
`socket.id`	Unique identifier of a socket
`socket.on("disconnect")`	Detect client disconnection

🧠 Real-Time Application Mental Model

Whenever you build a real-time application, think:



For this chess project:

```
Drag Piece
    ↓
Create Move
    ↓
socket.emit()
    ↓
Server
    ↓
Validate Move
    ↓
io.emit()
    ↓
All Players
    ↓
Update Board
```

🚀 Key Takeaways

Remember these points:

1. WebSocket

A protocol for persistent, two-way communication between client and server.

2. Socket.IO

A library that provides convenient event-based real-time communication and additional features around transports such as WebSocket.

3. `emit()`

Used to send an event.

```
socket.emit("move", data);
```

4. `on()`

Used to listen for an event.

```
socket.on("move", callback);
```

5. `io.emit()`

Broadcasts to everyone.

```
io.emit("move", data);
```

6. `socket.broadcast.emit()`

Broadcasts to everyone except the sender.

7. `socket.id`

Identifies a connected client.

8. Server as Source of Truth

Never rely completely on client-side validation for important game logic.

9. Events

Real-time applications are commonly designed around:

```
Event → Listener → Handler
```

10. State Synchronization

All clients need to stay synchronized with the authoritative server state.